

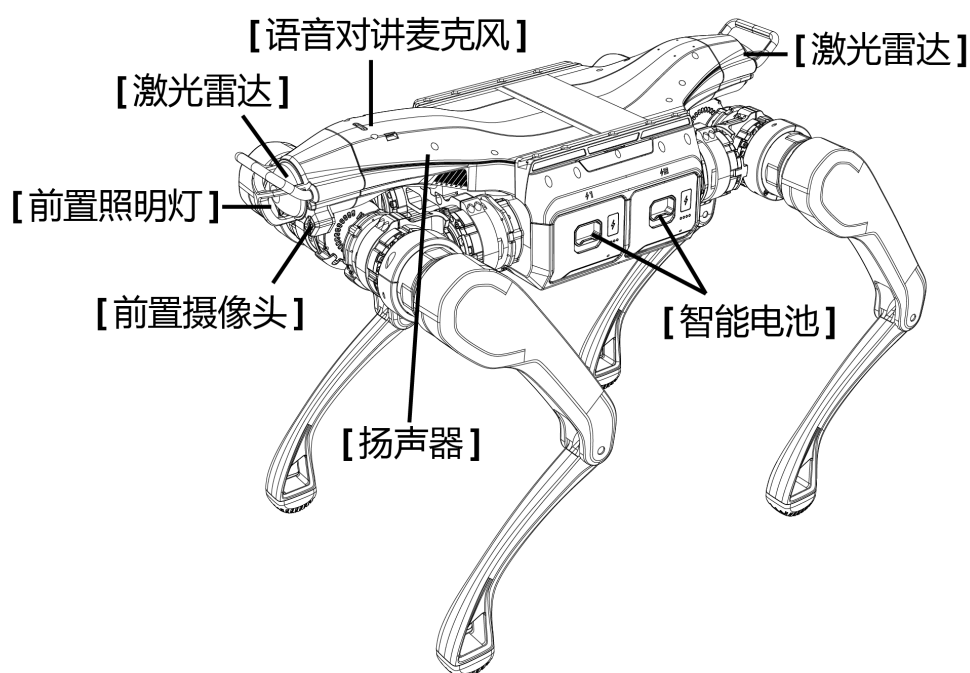
多媒体服务接口

来源: https://support.unitree.com/home/zh/A2_SDK_Development_Guide/vui_service

多媒体服务接口

更新时间: 2026-05-25 17:06:16

介绍



A2内嵌的音频/灯光相关的硬件设施, 使得机器人本体具备了实现语音交互的能力, 包括:

- 灯条 (绿色与白色)
- 扬声器
- 麦克风

二次开发

软件服务版本要求: Vui Service > 1.253.0.3。若内置服务版本不符合此要求, 请联系技术支持升级至正确版本。

音频灯光相关接口目前主要提供以下能力, 用户可以使用以下组合来开发自己的语音交互程序。

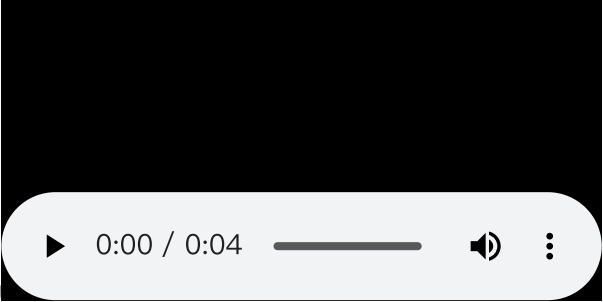
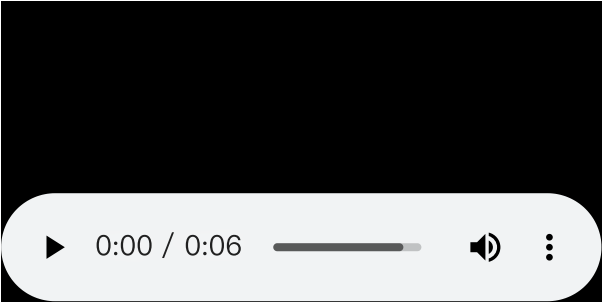
- ASR(Automatic Speech Recognition语音识别), 默认非流式模型, 本地离线式。输出结果包含方位角, 情感, 说话人角色等信息, 部分字段在开发中, 请注意官网更新。
- TTS(Text-to-Speech 文本转语音), 本地离线合成, 发音角色可选, 支持中文和英文两种语言
- 音频流控制

- 音量控制
- 灯带颜色控制

AudioClient 类

AudioClient类可以实现文本转语音，音频控制/播放，灯光控制等功能。

接口列表：

函数名	TtsMaker
函数原型	int32_t TtsMaker(const std::string& text, int32_t speaker_id)
功能概述	文字转语音
参数	text : 文字 speaker_id : 角色id
返回值	调用成功返回 0，否则返回相关错误码。
备注	speaker_id中文角色填0 英文角色填1，暂不支持中英文混合模式。
speaker_id 0	 你好。我是宇树科技的机器人。例程启动成功
speaker_id 1	Hello. I'm a robot from Unitree Robotics. The example has started successfully. 

函数名	GetVolume
函数原型	int32_t GetVolume(uint8_t &volume)
功能概述	获取系统音量
参数	volume : 音量大小(0-100)
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	SetVolume
函数原型	int32_t SetVolume(uint8_t volume)
功能概述	设置系统音量
参数	volume : 音量大小(0-100)
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	LedControl
函数原型	int32_t LedControl(uint8_t R, uint8_t G, uint8_t B)
功能概述	灯带控制
参数	R : 红色(0-255) G : 绿色(0-255) B : 蓝色(0-255)
返回值	调用成功返回 0，否则返回相关错误码。
备注	此接口调用间隔需大于200ms

函数名	PlayStream
函数原型	int32_t PlayStream(std::string app_name, std::string stream_id, std::vector<uint8_t> pcm_data)
功能概述	音频流播放
参数	app_name : 应用名称 stream_id : 标识id, 相同为缓存续播，不同则打断播放 pcm_data : PCM格式 采样率16K 单通道 16bit
返回值	调用成功返回 0，否则返回相关错误码。
备注	注意音频格式

函数名	PlayStop
函数原型	int32_t PlayStop(std::string app_name)
功能概述	停止播放
参数	app_name : 应用名称
返回值	调用成功返回 0，否则返回相关错误码。
备注	

ASR消息 / 播放状态

在麦克风打开状态下(可使用APP或者遥控器切换到唤醒模式)，内置麦克风+ASR模块会识别环境中的人声并进行识别。

订阅话题 rt/audio_msg(类型: std_msgs::msg::dds_::String_) 获取内置离线式ASR模块提供的识别信息

ASR消息

```
{
  "index": 1,
  "timestamp":29319303490
  "text": "你好",
  "angle": 90,
  "speaker_id": 0,
  "sense": "unknown",
  "confidence": 0.95,
  "language": "zh-CN",
  "is_final": true
}
```

参数名	参数类型	含义
index	整形	消息唯一序号
timestamp	整形	时间戳
text	字符串	语音识别结果
angle	整形	方位角度 0-180
speaker_id	整形	说话人识别结果
sense	字符串	情绪识别结果
confidence	浮点型	置信度
language	字符串	语言种类
is_final	整形	结束标志位 (用于流式识别模式，默认非流式)

播放状态

```
{
  "play_state": 1
}
```

参数名	参数类型	含义
play_state	整形	0:停止播放 1:开始播放

麦克风音频

在麦克风打开状态下(可使用APP或者遥控器切换到唤醒模式)，在组播地址239.168.123.161 端口5555会输出降噪后的音频数据。

音频数据格式为单通道/16K采样率/16bit

图像获取接口

接口说明

JPEG 拍照服务接口

利用该接口，发送方发送拍照请求，随后获得响应，即JPEG照片。具体实践是通过 VideoClient 类完成，VideoClient 类是 Video 服务提供的 Client，VideoClient 类可以通过 RPC 方式实现对 Go2 图像的获取。

h264 图传服务接口

在接收端，如需直接查看图传视频流，可以通过在命令行中输入如下指令实现：

```
gst-launch-1.0 udpsrc address=230.1.1.1 port=1720 multicast-iface=<interface_name> !
queue !  application/x-rtp, media=video, encoding-name=H264 ! rtph264depay ! h264parse

! avdec_h264 ! videoconvert ! autovideosink
```

其中 230.1.1.1是多播地址。
<interface_name> 是用户主机连接 Go2 的网络接口名称，例如 eth0， 设置成 multicast-iface=eth0 即可。

目前只支持 UDP，未来会加入其他图传协议的支持。
视频帧率为15Hz。相机分辨率为1280*720，水平视角100度，垂直视角56度。

JPEG 拍照服务接口

VideoClient 类

接口错误码列表：
暂无

类与接口描述：

类名	构造与析构
VideoClient	VideoClient::~VideoClient()

函数名	GetImageSample
函数原型	int32_t GetImageSample(std::vector<uint8_t>& image_sample)
功能概述	获取照片
参数	image_sample：照片内容
返回值	调用成功返回 0，否则返回相关错误码。
备注	

案例

本案例展示如何通过JPEG拍照服务接口获取图片并保存。

```

#include <unitree/robot/go2/video/video_client.hpp>
#include <iostream>
#include <fstream>
#include <ctime>

int main(int argc, char **argv)
{
    /*
     * Initilaize ChannelFactory
     */
    if (argc < 2)
    {
        std::cout << "Usage: " << argv[0] << " networkInterface" << std::endl;
        exit(-1);
    }
    unitree::robot::ChannelFactory::Instance()->Init(0, argv[1]);
    unitree::robot::go2::VideoClient video_client;

    /*
     * Set request timeout 1.0s
     */
    video_client.SetTimeout(1.0f);
    video_client.Init();

    //Test Api

    std::vector<uint8_t> image_sample;
    int ret;

    while (true)
    {
        ret = video_client.GetImageSample(image_sample);

        if (ret == 0) {
            time_t rawtime;
            struct tm *timeinfo;
            char buffer[80];

            time(&rawtime);
            timeinfo = localtime(&rawtime);

            strftime(buffer, sizeof(buffer), "%Y%m%d%H%M%S.jpg", timeinfo);

```

```

        std::string image_name(buffer);

        std::ofstream image_file(image_name, std::ios::binary);
        if (image_file.is_open()) {
            image_file.write(reinterpret_cast<const char*>(image_sample.data()),
image_sample.size());
            image_file.close();
            std::cout << "Image saved successfully as " << image_name << std::endl;
        } else {
            std::cerr << "Error: Failed to save image." << std::endl;
        }
    }

    sleep(3);
}

return 0;
}

```

h264 图传服务接口

图传推荐使用 Gst 的定向 UDP 接口。若您打算使用 Gst 来拉流，可参考以下内容，实现在 OpenCV 中使用 Gst 进行拉流。

Gst 支持嵌入 OpenCV 的 VideoCapture 函数中以 Gst pipeline 的方法进行拉流。同时它也支持嵌入 **VideoWriter** 函数中以 Gst pipeline 的方式进行推流。

依赖相关

Gst 安装

Ubuntu Linux 环境下安装：


```
sudo apt-get install libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev libgstreamer-  
plugins-bad1.0-dev gstreamer1.0-plugins-base gstreamer1.0-plugins-good gstreamer1.0-  
plugins-bad gstreamer1.0-plugins-ugly gstreamer1.0-libav gstreamer1.0-tools  
gstreamer1.0-x gstreamer1.0-alsa gstreamer1.0-gl gstreamer1.0-gtk3 gstreamer1.0-qt5
```

```
gstreamer1.0-pulseaudio
```

说明：

Windows 环境可以参考官方文档：<https://gstreamer.freedesktop.org/documentation/installing/on-windows.html?gi-language=c>

OpenCV 编译

帮助

如果无法使用cmake gui、出现无法使用configure的情况，或需要使用Python，可参考[帮助中心](#)，使用命令行编译的方法安装。

为了实现上述功能，需要在编译 OpenCV 时勾选 **WITH_GSTREAMER** 选项。OpenCV源码编译过程请参考以下步骤：

1、Github 中下载 OpenCV4.1.1源码 <https://github.com/opencv/opencv/tree/4.1.1>

2、安装

```
mkdir build
cd build
cmake-gui ..

####此时进入gui对话框
####点击configure
####勾选 with cuda 和 with Gstreamer
####再次点击configure
####之后点击generate
####关闭gui对话框

make

sudo make install
```

3、OpenCV 中可以通过调用 `getBuildInformance` 函数查看 OpenCV 的编译情况，具体如下所示：

```
#include <opencv2/opencv.hpp>

int main(void)
{
    std::cout << cv::getBuildInformation() << std::endl;

}
```

4、通过运行上述函数，可以得到如下输出，其中 Video I/O 中可以查看 Gstreamer 选项是否打开。

```
General configuration for OpenCV 4.5.1 =====
Version control:          unknown

Extra modules:
  Location (extra):
  Version control (extra): unknown

Platform:
  Timestamp:              2022-11-03T02:40:40Z
  Host:                   Linux 5.4.0-42-generic x86_64
  CMake:                  3.22.2
  CMake generator:        Unix Makefiles
  CMake build tool:        /usr/bin/make
  Configuration:          Release

Video I/O:
  FFMPEG:                 YES
    avcodec:              YES (57.107.100)
    avformat:             YES (57.83.100)
    avutil:               YES (55.78.100)
    swscale:              YES (4.8.100)
    avresample:           YES (3.7.0)
  GStreamer:              YES (1.14.5)
  v4l/v4l2:               YES (linux/videodev2.h)
```

视频拉流

OpenCV 的 **VideoCapture** 类支持视频文件，图像序列或者摄像头作为输入，获取数据。通过调用如下的 **VideoCapture** 构造函数能实现对 USB 相机数据流的拉取：

```
cv::VideoCapture::VideoCapture(const String & filename,int apiPreference)
```

变量	说明
filename	该参数可以是以下几类： * 视频文件路径； * 图像序列，例如图像序列名为 image_00.jpg, image_01.jpg, image_02.jpg, 则可以设置为 image_%02d.jpg； * 视频流的 URL，例如：rtsp://admin@password:192.168.170.XXX； * Gstreamer 的 pipeline string ，该 pipeline 可以使用 gst-launch-1.0 测试可用性；
apiPreference	Capture API 使用何种后端，包括 CAP_GSTREAMER or CAP_FFMPEG or CAP_IMAGES or CAP_DSHOW 等。

参考案例

若您使用 C++ ，可参考如下代码

```

#include <opencv2/opencv.hpp>
using namespace cv;

#include <iostream>
using namespace std;

int main()
{

    VideoCapture cap("udpsrc address=230.1.1.1 port=1720 multicast-iface=<interface_name>
! application/x-rtp, media=video, encoding-name=H264 ! rtph264depay ! h264parse !
avdec_h264 ! videoconvert ! video/x-raw,width=1280,height=720,format=BGR ! appsink
drop=1",
                    CAP_GSTREAMER);

    if (!cap.isOpened()) {
        cerr <<"VideoCapture not opened"<<endl;
        exit(-1);
    }

    while (true) {

        Mat frame;

        cap.read(frame);

        imshow("receiver", frame);

        if (waitKey(1) == 27) {
            break;
        }
    }

    return 0;

}

```

若您使用 Python，可参考如下代码

```
import cv2
gststreamer_str = "udpsrc address=230.1.1.1 port=1720 multicast-iface=<interface_name> !
application/x-rtp, media=video, encoding-name=H264 ! rtph264depay ! h264parse !
avdec_h264 ! videoconvert ! video/x-raw,width=1280,height=720,format=BGR ! appsink
drop=1"
cap = cv2.VideoCapture(gststreamer_str, cv2.CAP_GSTREAMER)
while(cap.isOpened()):
    ret, frame = cap.read()
    if ret:
        cv2.imshow("Input via Gstreamer", frame)
        if cv2.waitKey(25) & 0xFF == ord('q'):
            break
    else:
        break
cap.release()

cv2.destroyAllWindows()
```